Computational Economics
https://doi.org/10.1007/s10614-024-10797-w

Forecasting Crude Oil Prices Using Reservoir Computing Models

Kaushal Kumar[1]

Accepted: 10 November 2024

Abstract

Accurate forecasting of crude oil prices is crucial for informed financial decision-making. This study presents a cutting-edge Reservoir Computing (RC) model specifically designed for precise crude oil price predictions, outperforming traditional methods such as ARIMA, LSTM, and GRU. Using daily closing prices from major indices spanning January 2010 to December 2023, we conducted a thorough evaluation. The RC model consistently demonstrates superior accuracy and computational efficiency. Quantitative metrics reveal the RC model's dominance with a Mean Absolute Error (MAE) of 0.0094, Mean Squared Error (MSE) of 0.00035, Root Mean Squared Error (RMSE) of 0.0196, and a notably low Mean Absolute Percentage Error (MAPE) of 1.450%. Additionally, the RC model's runtime of 1.11 s underscores its computational efficiency, far surpassing ARIMA (493.22 s), LSTM (423.55 s), and GRU (15.73 s). During periods of economic disruption, such as the COVID-19 lockdowns, the RC model effectively captured sharp price fluctuations, highlighting its robust forecasting capability. These findings emphasize the RC model's potential as a reliable tool for enhancing decision-making processes in the dynamic energy market, particularly for real-time applications such as infectious disease case count forecasting. This study advocates for the broader adoption of Reservoir Computing models to improve predictive accuracy and operational efficiency in energy economics.

Keywords Crude oil prices · Time series analysis · Reservoir computing · Financial market prediction · Forecasting

✉ Kaushal Kumar
kaushal.kumar@stud.uni-heidelberg.de

[1] Institute for Mathematics, Heidelberg University, Im Neuenheimer Feld 205, Heidelberg 69120, Baden-Württemberg, Germany

Published online: 28 November 2024

1 Introduction

Accurately predicting financial market prices, particularly crude oil prices, is crucial for effective decision-making and policy formulation. Machine learning and deep learning methods have become pivotal tools in this domain, leveraging historical time series data for predictive analytics (Sezer et al., 2020; Hamayel & Owda, 2021). Despite advancements, precise financial price prediction remains challenging, prompting the exploration of innovative approaches.

Traditional econometric models often face limitations in capturing the intricate and nonlinear patterns within financial time series. In contrast, machine learning techniques offer promising avenues for uncovering hidden relationships within extensive datasets. This study introduces a novel machine learning model rooted in reservoir computing-a variant of recurrent neural networks known for its ability to capture temporal dependencies and nonlinear dynamics in time series data.

Reservoir Computing (RC) stands out due to its unique architecture, which involves a fixed, randomly connected reservoir of neurons Lukoševičius and Jaeger (2009), Jaeger (2021, 2007), Wikipedia contributors (2023). This structure enables efficient handling of high-dimensional data and complex temporal patterns, offering significant advantages over traditional deep learning methods like LSTM and GRU.

The primary objective of this study is to evaluate the performance of the RC model in predicting crude oil prices. We conduct a comprehensive comparative analysis using daily closing price data for historic crude oil prices, covering 3,521 trading days from January 1, 2010, to December 31, 2023 (see Table 1). Figure 1 illustrates the time series of daily crude oil prices. Our benchmark includes established methods such as AutoRegressive Integrated Moving Average (ARIMA), Long Short-Term Memory (LSTM) Hochreiter and Schmidhuber (1997), and Gated Recurrent Unit (GRU) models, to ascertain the competitiveness of the RC model in predicting crude oil prices.

This study aims to develop and evaluate a novel RC model tailored for financial market predictions, specifically enhancing crude oil price forecasting. By building on prior studies (Sheng et al., 2014; Maass et al., 2002; Doan et al., 2020), our research contributes to the existing literature by introducing RC as a promising approach for accurate and reliable time series data prediction.

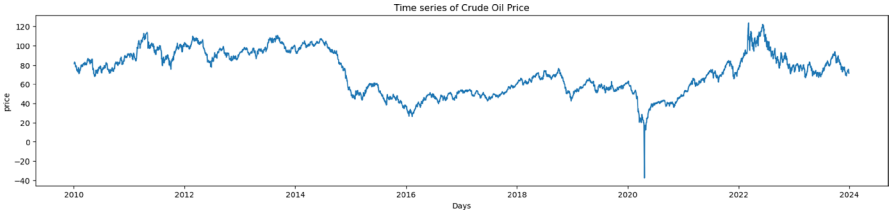


Fig. 1 The above plot show the time series of daily crude oil prices. The dataset comprises a collection of time series representing the daily closing price of historic crude oil prices, spanning the period from January 1, 2010, to December 31, 2023. There is a sharp fluctuation in oil prices between March and April 2020 during the Covid-19 pandemic (Le et al., 2021)

Table 1 The table shows the time series of daily crude oil prices

	Date	Open	High	Low	Close	Adj Close	Voume
0	2010-01-04	79.629997	81.680000	79.629997	81.510002	81.510002	263542
1	2010-01-05	81.629997	82.000000	80.949997	81.769997	81.769997	258887
2	2010-01-06	81.430000	83.519997	80.849998	83.180000	83.180000	370059
3	2010-01-07	83.199997	83.360001	82.260002	82.660004	82.660004	246632
4	2010-01-08	82.650002	83.470001	81.800003	82.750000	82.750000	310377
...	..	..	..	..	..	..	..
3516	2023-12-22	73.910004	74.980003	73.389999	73.559998	73.559998	222600
3517	2023-12-26	73.559998	76.180000	73.129997	75.570000	75.570000	208715
3518	2023-12-27	75.320000	75.660004	73.769997	74.110001	74.110001	253323
3519	2023-12-28	73.800003	74.400002	71.720001	71.769997	71.769997	262748
3520	2023-12-29	71.989998	72.620003	71.250000	71.650002	71.650002	214486

The dataset comprises a collection of time series representing the daily closing price of historic crude oil prices, spanning the period from January 1, 2010, to December 31, 2023

The structure of this paper is as follows:

1. *Literature Review* Discusses existing methodologies and their limitations in crude oil price forecasting.
2. *Methodology* Provides a detailed explanation of the RC model, including its theoretical background and implementation details.
3. *Results and Discussion* Presents the comparative performance of the RC model against ARIMA, LSTM, and GRU, using various evaluation metrics.
4. *Conclusion* Summarizes the findings, highlights the implications of the study, and suggests future research directions.

By integrating these elements, this study aims to contribute to the field of energy economics by providing a robust and efficient tool for crude oil price prediction.

2 Related Works

Predicting crude oil prices has been a long-standing research focus, traditionally relying on econometric models like AutoRegressive Integrated Moving Average (ARIMA). However, these models often struggle to capture complex and nonlinear patterns in financial data. The rise of machine learning (ML) and deep learning (DL) techniques has opened new avenues for more accurate and efficient predictions (Shih et al., 2019).

In deep learning, models such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRU) have gained popularity for their ability to capture temporal dependencies in sequential data. These models have demonstrated proficiency in learning complex patterns and nonlinear relationships in time series forecasting Rajagukguk et al. (2020), Weerakody et al. (2021), Kumar (2023). Studies

have shown their effectiveness in predicting stock prices, exchange rates, and commodity prices. However, LSTM and GRU models can be computationally intensive and require extensive hyperparameter tuning Goodfellow et al. (2016).

Reservoir Computing (RC) offers an alternative approach. RC, particularly the Echo State Network (ESN) variant, is a type of recurrent neural network (RNN) that leverages a fixed, randomly connected reservoir of neurons. This architecture transforms input signals into high-dimensional representations, which are then processed by a simple readout layer Jaeger and Haas (2004), Pathak et al. (2018), Jaeger (2002), Lukoševičius (2012). RC is known for its simplicity, computational efficiency, and ability to model complex temporal patterns without extensive parameter tuning Lukoševičius and Jaeger (2009).

Previous studies have successfully applied RC to various domains, including stock market prediction and epidemiological forecasting, demonstrating its versatility and robustness (Kumar, 2023; Wang et al., 2011). However, its application in crude oil price prediction remains underexplored. This study aims to fill this gap by evaluating the performance of an RC model in predicting daily crude oil prices, comparing it with established methods such as ARIMA, LSTM, and GRU.

By addressing the limitations of traditional and deep learning models, our research introduces RC as a viable and efficient alternative for time series forecasting. The findings are expected to enhance the understanding of RC's potential in financial market predictions and provide insights for future research.

2.1 Discussion on ARIMA, LSTM, GRU, and Reservoir Computing

2.1.1 ARIMA (AutoRegressive Integrated Moving Average)

Key Characteristics:

- **Type**: Linear statistical model.
- **Nature**: Non-sequential.
- **Components**:
  - AutoRegressive (AR): Assumes a linear combination of past values.
  - Integrated (I): Involves differencing to achieve stationarity.
  - Moving Average (MA): Assumes a linear combination of past errors.

Strengths and Weaknesses:

- **Strengths**:
  - Well-suited for linear relationships.
  - Interpretability of model parameters.
- **Weaknesses**:
  - Assumes linear relationships.
  - Limited ability to capture complex patterns.

2.1.2 LSTM (Long Short-Term Memory)

Key Characteristics:

- **Type**: Neural network architecture.
- **Nature**: Sequential with long-term dependency handling.
- **Components**:
  - Memory Cells: Maintain and update cell state.
  - Input Gate: Controls information entry.
  - Forget Gate: Controls information to discard.
  - Output Gate: Controls information to output.

Strengths and Weaknesses:

- **Strengths**:
  - Excellent for long-term dependencies.
  - Suitable for complex, non-linear patterns.
- **Weaknesses**:
  - Computationally intensive.
  - Prone to overfitting with small datasets.

2.1.3 GRU (Gated Recurrent Unit)

Key Characteristics:

- **Type**: Neural network architecture (simplified compared to LSTM).
- **Nature**: Sequential, designed for simplifying LSTM.
- **Components**:
  - Update Gate: Controls information to update.
  - Reset Gate: Controls information to discard.

Strengths and Weaknesses:

- **Strengths**:
  - Simplified structure, faster training.
  - Effective in capturing long-term dependencies.
- **Weaknesses**:
  - May not perform as well as LSTM in precise long-term memory tasks.

2.1.4 Reservoir Computing

Key Characteristics:

- **Type**: Utilizes Echo State Network (ESN).
- **Nature**: Utilizes a fixed reservoir of neurons.
- **Components**:
  - Reservoir: Fixed set of recurrently connected neurons.
  - Readout Layer: Trainable layer transforming reservoir states to output.

Strengths and Weaknesses:

- **Strengths**:
  - Simple training procedure (readout layer only).
  - Efficient for certain time-series prediction tasks.
- **Weaknesses**:
  - Performance highly depends on reservoir quality.
  - May not perform well on tasks with complex dependencies.

3 Materials and Methods

3.1 Data Collection

Crude oil price data was extracted from Yahoo Finance using the `yfinance` Python package (Aroussi, 2021). The `yfinance` library is commonly utilized in the open-source community to retrieve historical market data from Yahoo! Finance. In this study, we retrieved crude oil price data from January 1, 2010, to December 31, 2023. The dataset includes daily price values, providing a comprehensive time series spanning over 13 years as shown in Table 1.

The availability of such extensive data allows for a thorough analysis of crude oil price trends, patterns, and relationships with other variables. It provides a robust foundation for conducting accurate and reliable forecasting models and evaluating their performance.

Figure 2 displays the time series plots of six key components of crude oil prices: Open, High, Low, Close, Adjusted Close, and Volume, from the provided dataset. Each subplot represents the variation of one component over time, with the x-axis showing the dates and the y-axis showing the respective values.

3.1.1 Nature of Crude Oil

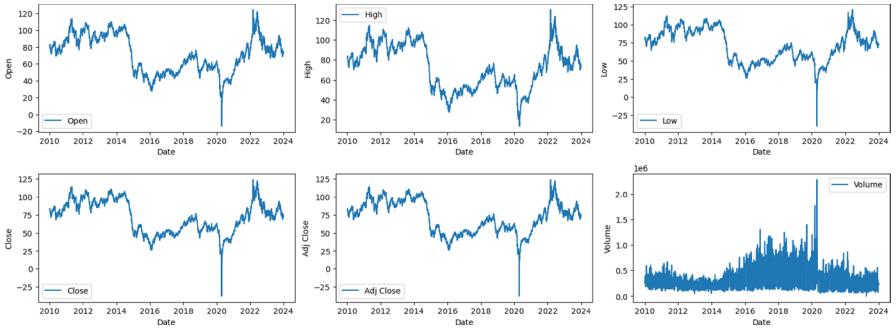


Fig. 2 Time Series Plots of Crude Oil Price Components

The mean and median prices (Open, High, Low, Close, Adj Close) are relatively close to each other, indicating that the distribution of prices is likely symmetric. The mean volume traded is higher than the median, suggesting that there might be some days with significantly higher trading volumes, skewing the distribution towards higher values.The standard deviation for price columns is around 22$, indicating moderate variability in prices. For the Volume column, the standard deviation is relatively large compared to the mean, indicating high variability in trading volumes. Negative skewness in all columns suggests a slight left skewness, implying that the distribution of prices and volume is slightly skewed towards lower values. The skewness value for the Volume column is higher than for the price columns, indicating a more pronounced skewness towards lower trading volumes. Negative kurtosis values for price columns indicate lighter tails compared to a normal distribution, suggesting that extreme price movements are less likely. The positive kurtosis value for the Volume column indicates heavier tails, implying that extreme trading volume values are more likely to occur as shown in Table 2.

These findings provide insights into the characteristics of the crude oil price data, which can be valuable for risk assessment, trading strategies, and investment decision-making in the crude oil market.

Table 2 Summary Statistics of Crude Oil Price Components

Statistic	Open	High	Low	Close	Volume
Mean	71.758770	72.814164	70.592318	71.729960	408349.433116
Median	72.180000	73.379997	71.089996	72.139999	354635.000000
Standard Deviation	22.002610	22.123886	21.865998	22.035948	206874.618995
Skewness	−0.058433	−0.031698	−0.103875	−0.077177	1.172312
Kurtosis	−0.974511	−1.029069	−0.799754	−0.875503	2.985762

3.2 Preprocessing

Before applying the prediction models, we performed data preprocessing steps to ensure the quality and suitability of the data. This included handling missing values, removing outliers, and normalizing the data to a consistent scale. Normalization is particularly important for machine learning algorithms to ensure fair comparisons and optimal performance.

To prepare the data for analysis, a normalization technique was employed, as outlined in Eq. (1). By applying this technique, the data were transformed to a range of values between 0 and 1, providing a standardized basis for comparison.

$$Y_{norm} = \frac{Y_i - Y_{min}}{Y_{max} - Y_{min}} \tag{1}$$

Here, Y_{norm} represents the normalized data, while Y_i, Y_{min}, and Y_{max} are the observed, minimum, and maximum values of crude oil prices per day, respectively.

3.3 Neural Network

A short mathematical detail of how neural networks work is explained in the following subsection (Haykin, 2009).

3.3.1 Perceptron

A perceptron takes multiple input values $x_1, x_2, ..., x_n$, each multiplied by a corresponding weight $w_1, w_2, ..., w_n$. The weighted inputs are summed, and a bias term b is added:

$$z = \sum_{i=1}^{n} (x_i \cdot w_i) + b \tag{2}$$

The result z is then passed through an activation function f to produce the output a of the perceptron:

$$a = f(z) \tag{3}$$

Common activation functions include the step function, sigmoid, hyperbolic tangent (tanh), and rectified linear unit (ReLU).

3.3.2 Neural Network Architecture

Neural networks consist of layers of perceptrons, typically organized into an input layer, one or more hidden layers, and an output layer. The connections between neurons are represented by weights.

For a simple feedforward neural network, the output a_j^l of a neuron j in layer l is computed as follows:

$$a_j^l = f\left(\sum_i (a_i^{l-1} \cdot w_{ij}^l) + b_j^l\right) \tag{4}$$

Here, a_i^{l-1} is the output of neuron i in the previous layer, w_{ij}^l is the weight connecting neuron i to neuron j, b_j^l is the bias term for neuron j in layer l, and f is the activation function.

3.3.3 Training a Neural Network

The training process involves adjusting the weights and biases to minimize the difference between the predicted output and the actual output (target). This is achieved through backpropagation and gradient descent.

1. Forward Pass: Compute the predicted output by propagating the input through the network.
2. Compute Loss: Compare the predicted output to the actual output using a loss function (e.g., mean squared error).
3. Backpropagation: Calculate the gradients of the loss with respect to the weights and biases (Rumelhart et al., 1986).
4. Gradient Descent: Update the weights and biases in the direction that reduces the loss, typically proportional to the negative gradient (Robbins, 1951).
5. Repeat: Iterate through the dataset multiple times (epochs) to refine the network's parameters.

Through this iterative process, the neural network learns to map input data to the desired output, adapting its weights and biases to capture underlying patterns in the training data.

3.4 Methods

3.4.1 ARIMA Model

Time series analysis is a powerful tool for modeling and forecasting sequential data points. The Autoregressive Integrated Moving Average (ARIMA) model is a widely-used method for time series forecasting (Tseng et al., 2002; Siami-Namini et al., 2018; Zhang, 2003). The ARIMA model is represented as ARIMA(p, d, q), with:

- p: Order of the autoregressive (AR) component,
- d: Degree of differencing,
- q: Order of the moving average (MA) component.

The general form of an ARIMA model is given by the equation:

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \ldots + \phi_p Y_{t-p} + \epsilon_t - \theta_1 \epsilon_{t-1} - \theta_2 \epsilon_{t-2} - \ldots - \theta_q \epsilon_{t-q} \quad (5)$$

where:

- Y_t: The observed time series at time t,
- $\phi_1, \phi_2, \ldots, \phi_p$: Coefficients of the autoregressive terms,
- ϵ_t: White noise error term at time t,
- $\theta_1, \theta_2, \ldots, \theta_q$: Coefficients of the moving average terms.

Predicting crude oil prices is a complex task due to the multifaceted nature of the oil market, influenced by geopolitical, economic, and environmental factors. The ARIMA model provides a framework for time series forecasting, and its application to crude oil price prediction involves several key steps.

3.4.2 Long Short-Term Memory (LSTM)

The Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) designed to capture long-term dependencies in sequential data (Elsworth & Güttel, 2020; Zhang et al., 2019; Goodfellow et al., 2016; Nakajima, 2020). The mathematical details involve memory cells, input, forget, and output gates. The architecture of the LSTM is depicted in Fig. 3.

Memory Cell and Gates: The memory cell, c_t, in an LSTM is updated using input, forget, and output gates.

Input Gate: The input gate, i_t, is computed using a sigmoid activation function:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (6)$$

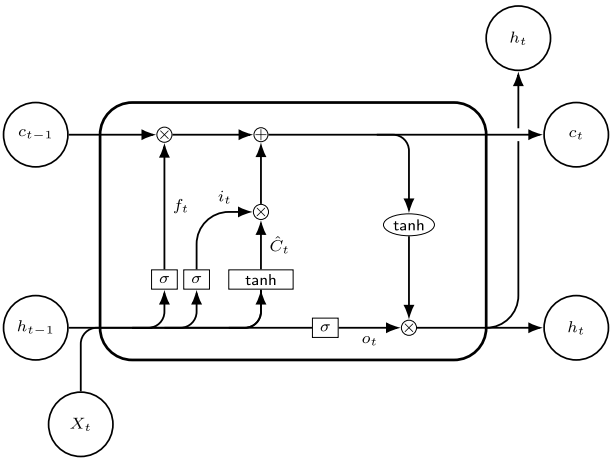


Fig. 3 Structure of a LSTM

where W_i is the weight matrix, b_i is the bias term, and σ is the sigmoid activation function. $[h_{t-1}, x_t]$ represents the concatenation of the previous hidden state and the current input.

Forget Gate: The forget gate, f_t, is computed using a sigmoid activation function:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{7}$$

where W_f is the weight matrix, b_f is the bias term.

Memory Cell Update: The new memory cell, $\tilde{c}_t$, is computed using the hyperbolic tangent (tanh) activation function:

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \tag{8}$$

where W_c is the weight matrix, b_c is the bias term.

Final Memory Cell Update: The memory cell undergoes updates influenced by both the input and forget gates:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{9}$$

Output Gate: The output gate, o_t, is computed using a sigmoid activation function:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{10}$$

where W_o is the weight matrix, b_o is the bias term.

Hidden State Update: The hidden state, h_t, is computed using the output gate and the memory cell:

$$h_t = o_t \odot \tanh(c_t) \tag{11}$$

Output: The output y_t at each time step can be obtained by further processing the hidden state or directly using it, depending on the specific architecture and requirements of the model.

Training: The model is trained by adjusting the weights W_i, W_f, W_c, and W_o, as well as the bias terms b_i, b_f, b_c, and b_o through backpropagation (Rumelhart et al., 1986) and optimization algorithms like stochastic gradient descent (SGD) (Ruder, 2017; Bottou, 2010).

3.4.3 Gated Recurrent Unit (GRU)

The Gated Recurrent Unit (GRU) is a type of recurrent neural network (RNN) that is well-suited for time series forecasting. The mathematical details of a GRU model involve update gates, reset gates, and candidate hidden states (Time-series production, 2022; Tao et al., 2019; Li et al., 2020; Hamayel & Owda, 2021). The architecture of the GRU is depicted in Fig. 4.

Update and Reset Gates: The update gate, z_t, and reset gate, r_t, are computed using sigmoid activation functions:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \tag{12}$$

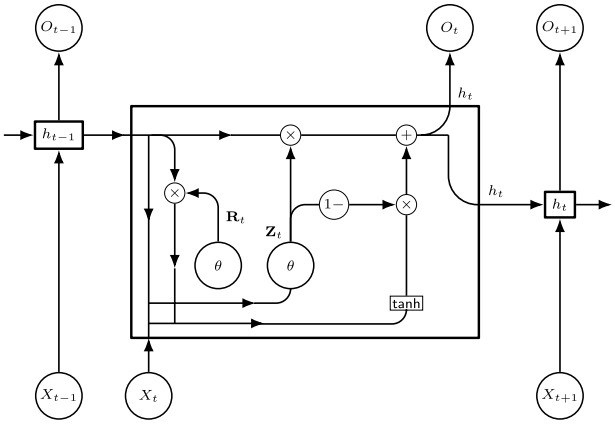


Fig. 4 Structure of a GRU network

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \tag{13}$$

where W_z, W_r, b_z, and b_r are weight matrices and bias terms, and σ is the sigmoid activation function. $[h_{t-1}, x_t]$ represents the concatenation of the previous hidden state and the current input.

Candidate Hidden State: A candidate hidden state, $\tilde{h}_t$, is computed using the hyperbolic tangent (tanh) activation function:

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h) \tag{14}$$

where W_h and b_h are the weight matrix and bias term, and $\odot$ represents element-wise multiplication.

Hidden State Update: The new hidden state, h_t, is a combination of the previous hidden state h_{t-1} and the candidate hidden state $\tilde{h}_t$ using the update gate z_t:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{15}$$

Output: The output y_t at each time step can be obtained by further processing the hidden state or directly using it, depending on the specific architecture and requirements of the model.

Training: The model is trained by adjusting the weights W_z, W_r, W_h, and bias terms b_z, b_r, b_h through backpropagation and optimization algorithms like stochastic gradient descent (SGD).

For crude oil price prediction, the input x_t could represent historical prices and relevant features. The model learns to capture patterns and dependencies in the historical data to make predictions about future prices.

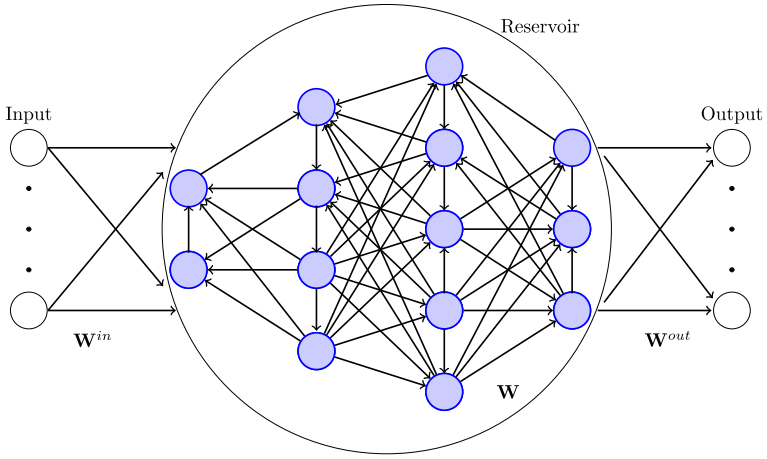


Fig. 5 Structure of an echo-state network (Reservoir Computing)

3.4.4 Reservoir Computing

In this section, we explore the application of reservoir computing methods for the prediction of crude oil prices. Reservoir computing is a machine learning technique that utilizes a fixed nonlinear dynamical system, known as a reservoir, to process temporal data and make predictions.

Reservoir computing methods, such as Echo State Networks (ESNs), are particularly effective in capturing complex temporal patterns in time series data like crude oil prices. The reservoir, consisting of a large number of interconnected nodes, acts as a computational resource that transforms the input data into a high-dimensional feature space. The architecture of the Reservoir computing is depicted in Fig. 5. For more detailed technical discussions on RC networks and Echo State Networks, one can refer to the seminal papers by Herbert Jaeger (Jaeger, 2001) and his collaborators, as well as reviews and tutorials that delve into the mathematical underpinnings and practical implementations of these networks (Nakajima, 2020; Hochreiter & Schmidhuber, 1997; Cucchi et al., 2022; Tanaka et al., 2019).

To apply reservoir computing for crude oil price prediction, we first preprocess the data and normalize it to a suitable range. This normalization step ensures that the data is within a consistent scale, facilitating the learning process of the reservoir computing model.

The mathematical details of the reservoir computing model involve the computation of the reservoir state dynamics and the output mapping. Let's denote the input sequence as $X = [x_1, x_2, ..., x_n]$, where x_i represents the i-th data point in the sequence. The reservoir state sequence is computed as $R = [r_1, r_2, ..., r_n]$, where r_i represents the state of the reservoir at time step i. The output sequence is derived from the reservoir state sequence.

In each time step $t + 1$, the Echo State Network (ESN) updates its state based on the input $x(t)$ using the following equation:

$$r(t+1) = (1-\alpha)r(t) + \alpha f(x(t+1), r(t)), \tag{16}$$

Here, α is a value between 0 and 1. The function $f(x,t)$ is defined as $tanh(W_{in}x + W_{res}.r)$, where W_{in} is the input weight matrix, and W_{res} is the reservoir weight matrix. The function f represents a non-linear activation function applied element-wise. The ESN predicts the target at time $t+1$ using the following equation:

$$\hat{y}(t+1) = [1; r(t+1)]W_{out}, \tag{17}$$

Here, W_{out} is the output weight matrix.

The weight matrices (W_{in}, W_{res}, and W_{out}) are learned during the training process using techniques like ridge regression or least squares optimization.

3.4.5 Model Evaluation

To assess the performance of the LSTM and reservoir computing models, We utilize various metrics to assess the performance of our model, which include the mean squared error (MSE), mean absolute error (MAE), root mean squared error (RMSE), mean absolute percentage error (MAPE), and Normalised RMSE (based on mean) (Hyndman & Athanasopoulos, 2018; Abraham & Ledolter, 1983). These metrics allow us to evaluate the accuracy and precision of our predictions. The definitions of these metrics can be defined as:

- Mean Absolute Error (MAE): MAE (Mean Absolute Error): This is the average of the absolute differences between predicted and actual values. It provides a measure of the average magnitude of errors without considering their direction.

$$MAE = \frac{1}{n}\Sigma_{i=1}^{n}|y_i - \hat{y}_i| \tag{18}$$

- Mean Squared Error (MSE): MSE (Mean Squared Error): This is the average of the squared differences between predicted and actual values. Like RMSE, it penalizes larger errors more than MAE.

$$MSE = \frac{1}{n}\Sigma_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{19}$$

- Root Mean Squared Error (RMSE): RMSE (Root Mean Squared Error): This is the square root of the average of the squared differences between predicted and actual values. It gives more weight to large errors than MAE.

$$RMSE = \sqrt{\frac{1}{n}\Sigma_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{20}$$

- Mean Absolute Percentage Error (MAPE): MAPE (Mean Absolute Percentage Error): This is the average of the absolute percentage differences between predicted and actual values. It is expressed as a percentage and provides a relative measure of the accuracy of predictions.

$$MAPE = \frac{1}{n}\Sigma_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100 \tag{21}$$

- Normalized RMSE (based on the mean): NRMSE (Normalized Root Mean Squared Error): This is the RMSE normalized by the range of the actual values. It provides a normalized measure of the error, making it easier to interpret across different datasets.

$$NRMSE_{mean} = \frac{RMSE}{mean(y)} \tag{22}$$

where: n is the number of samples, y_i is the actual value of the ith sample, $\hat{y}_i$ is the predicted value of the ith sample.

4 Results

4.1 Implementation Details

In order to ensure the effectiveness and applicability of the system, we carefully divided the dataset into two main subsets: a training set and a validation set. The training set served as the foundation for training the system, while the validation set was crucial for evaluating its performance on data it had not seen before. This careful separation was implemented to prevent overfitting, ensuring that the system learned genuinely and could generalize well to new data.

The dataset was thoughtfully divided, with 75% allocated for training and 25% for validation, aiming to strike a balance that would guarantee the validity and reliability of the obtained results. By training the models on the training set and subsequently assessing their performance on the testing set, we aimed to gauge their ability to generalize beyond the specific data they were trained on.

4.1.1 Programming Environment

All experiments and analyses were conducted using the Python programming language. Popular libraries such as `Scikit`, `yfinance` and `ReservoirPy` were utilized for implementing ARIMA, LSTM, GRU, and reservoir computing models (Trouvain et al., 2020). The models were trained and evaluated on a CPU with 32 GB RAM, featuring an Intel®Core™ i7-9700 CPU@ 3.00GHz × 8 to ensure efficient computation.

4.1.2 ARIMA Model

For the ARIMA model, parameters (p, d, q) are manually set to (2, 1, 1) in this simulation. Optimization techniques like grid search or more sophisticated algorithms (e.g., Bayesian optimization) can be employed to find the best set of parameters.

4.1.3 LSTM and GRU Models

In the case of LSTM and GRU models, we structured the model as a Sequential neural network with two main layers. The first layer of both LSTM and GRU consists of 50 units, designed to capture long-term dependencies in time series data. The second layer is a Dense output layer with a single unit, representing the model's output. During training, we used the Adam optimizer (Kingma & Adam, 2017), a widely-used optimization algorithm, and the Mean Squared Error (MSE) as the loss function. The model underwent 40 epochs, updating weights based on batches of 32 samples in each epoch. These settings define the architecture and training parameters crucial for the time series prediction task. Adjustments to these parameters can be explored to enhance model performance based on your specific dataset characteristics.

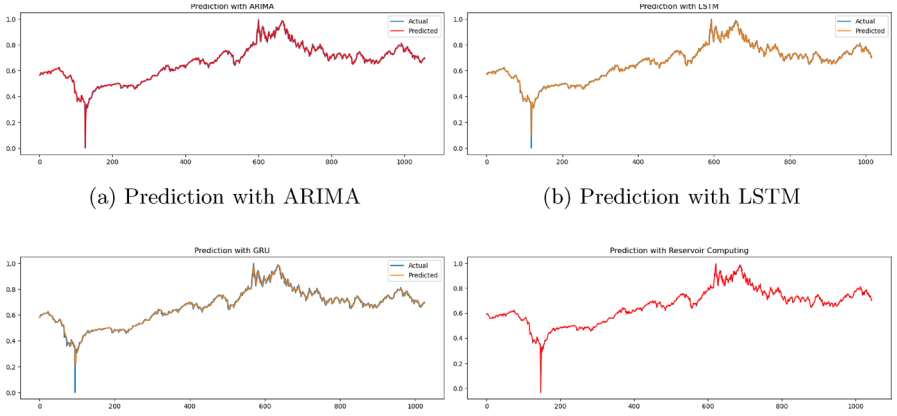
4.1.4 Reservoir Computing

In the case of reservoir computing, a model is implemented with specific parameters: 20 units, a leak rate of 0.75, spectral radius ρ of 1.025, input scaling of 1.0, reservoir connectivity of 0.15, input connectivity of 0.2, feedback connectivity of 1.1, and a regularization coefficient of 1×10^{-8}. Hyperparameter optimization (Yperman & Becker, 2017) is facilitated by the Hyperopt library, utilizing the Tree of Parzen Estimators (TPE) algorithm (Ren & Ma, 2022). The objective function for optimization involves training the reservoir model with varying hyperparameters (Valencia et al., 2023) and minimizing a performance metric, such as Mean Squared Error (MSE), on a validation set. The search space for hyperparameters includes choices for the number of units, leak rate, reservoir and input connectivity, and regularization coefficient. The optimized hyperparameters are then printed, providing an automated method for tuning the reservoir model's parameters for improved performance.

4.2 Statistical Analysis

In evaluating the performance of reservoir computing models, a comprehensive comparison against traditional time series forecasting methodologies such as ARIMA, LSTM, and GRU was conducted to assess their relative efficacy in predicting daily crude oil prices. The comparison was based on a range of evaluation metrics as depicted in Table 3, offering insightful observations:

- Reservoir Computing demonstrates superior predictive performance compared to ARIMA, LSTM, and GRU across multiple metrics including MAE, MSE, RMSE, MAPE, and normalized RMSE. These metrics collectively highlight the model's ability to achieve lower error rates and higher accuracy in forecasting crude oil prices.



(a) Prediction with ARIMA (b) Prediction with LSTM

(c) Prediction with GRU (d) Prediction with Reservoir Computing

Fig. 6 The above plots show the time series prediction of daily crude oil price using different approaches

Table 3 Evulation Matrices

Methods	**ARIMA**	**LSTM**	**GRU**	**Reservoir Computing**
MAE	0.0096	0.0138	0.0107	0.0094
MSE	0.00037	0.0004	0.00035	0.0004
RMSE	0.0198	0.0223	0.0198	0.0196
MAPE	6.2665%	2.7686%	1.6025%	1.450%
Normalized RMSE	0.0288	0.0337	0.0281	0.0314
Run time(in seconds)	493.22	423.55	15.73	1.11

- Notably, Reservoir Computing exhibits a significantly lower MAPE of 1.5705% in contrast to the alternative methodologies, signifying its proficiency in providing more precise percentage predictions.
- An analysis of computational efficiency reveals that Reservoir Computing boasts a markedly shorter runtime of 1.11 s, considerably outpacing ARIMA (493.22 s), LSTM (423.55 s), and GRU (15.73 s). This underscores Reservoir Computing's computational prowess and efficiency in handling large-scale datasets.

In summary, the evaluation metrics underscore Reservoir Computing's superior predictive capabilities over ARIMA, LSTM, and GRU in forecasting crude oil prices. With lower error rates, higher accuracy, and enhanced computational efficiency, Reservoir Computing emerges as a promising approach for time series forecasting tasks.

Figure 6 visually depicts the prediction results, with the blue line representing actual crude oil prices, while the orange and red lines denote predictions generated by LSTM and Reservoir Computing, respectively. Notably, Reservoir Computing

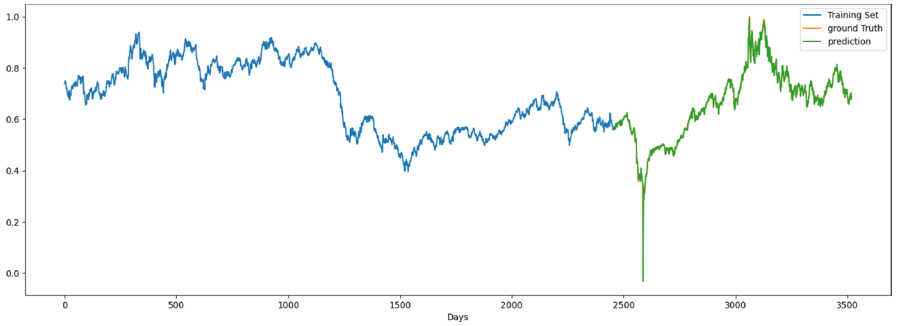


Fig. 7 The above plots shows the time series prediction of the daily closing price of historic crude oil prices, spanning the period from January 1, 2010, to December 31, 2023, using the Reservoir Computing approach

captures sharp fluctuations more accurately, as evidenced by the comparative analysis.

Moreover, Fig. 7 illustrates the entirety of the datasets, with the blue line denoting the training set, the orange line representing ground truth (test data), and the green line depicting predictions utilizing reservoir computing methodologies. This visualization provides further insights into the predictive performance of Reservoir Computing in capturing underlying patterns and trends within the crude oil price data.

5 Conclusions

This study presented a comprehensive evaluation of various machine learning and deep learning algorithms, specifically ARIMA, LSTM, GRU, and Reservoir Computing (RC), for forecasting historical crude oil prices. Our analysis identified Reservoir Computing as the superior model, consistently demonstrating higher prediction accuracy and greater computational efficiency.

In the quantitative assessment, RC outperformed other methods across multiple evaluation metrics, including Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and normalized RMSE. The RC model consistently exhibited lower error values and higher accuracy, particularly achieving significantly lower MAPE, which indicates more precise percentage predictions compared to alternative methods.

Qualitatively, RC's robustness in capturing complex patterns and handling noisy data was evident, especially during periods of economic disruption such as the COVID-19 lockdowns. It successfully predicted sharp drops in oil prices, highlighting its robust forecasting capabilities.

Additionally, RC demonstrated a substantial advantage in computational efficiency. Its faster runtime compared to other models underscores its potential suitability for real-time or time-sensitive applications, making it an attractive option for dynamic market environments.

While our findings strongly support the efficacy of RC for crude oil price prediction, it is important to recognize potential limitations, such as model interpretability and data requirements, which may affect its broader applicability. Moreover, qualitative assessments beyond statistical comparisons, such as the model's adaptability to various market conditions and long-term stability, need further exploration. Addressing these aspects would provide a more comprehensive understanding of RC's practical utility.

Future research directions could include refining RC models, exploring ensemble methods, incorporating additional data sources, and addressing computational challenges. Expanding on potential future directions or specific areas for improvement in Reservoir Computing would enhance the conclusion's depth and completeness. Such efforts can further enhance RC's predictive capabilities and broaden its application scope in energy economics and other fields.

In conclusion, our study, "Forecasting Crude Oil Prices Using Reservoir Computing Models," advocates for the adoption of Reservoir Computing as a reliable and efficient approach for crude oil price prediction. Recognizing its limitations while appreciating its strengths, RC holds significant potential for advancing forecasting methodologies and informing strategic decisions in dynamic markets.

Acknowledgements I would like to express my gratitude to my colleagues for their support, insightful discussions, and valuable contributions that have significantly influenced and strengthened this research. Additionally, I extend thanks to the anonymous reviewer for their valuable suggestions.

Funding Open Access funding enabled and organized by Projekt DEAL. This study is conducted without any financial support.

Data Availibility The code and data associated with this work will be made openly accessible on GitHub-https://github.com/kaushalkumarsimmons/Forecasting-Crude-Oil-Prices-Using-Reservoir-Computing-Models upon acceptance of the manuscript. The datasets were obtained using Yahoo Finance.

Declarations

Conflict of interest I have no conflict of interest or conflict of interest to disclose.

Ethical Approval This study involves the analysis of publicly available financial data and does not involve any human subjects or sensitive information. Therefore, no specific ethical considerations were required for this research.

References

Abraham, B., Ledolter, J. (1983). Statistical methods for forecasting. https://api.semanticscholar.org/CorpusID:61135356

Aroussi, R. (2021). yfinance. https://github.com/ranaroussi/yfinance. Accessed: 31/05/2023.

Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. In Lechevallier, Y., Saporta, G. (eds.) *Proceedings of COMPSTAT'2010,* pp. 177–186. Physica-Verlag HD, Heidelberg.

Cucchi, M., Abreu, S., Ciccone, G., Brunner, D., & Kleemann, H. (2022). Hands-on reservoir computing: A tutorial for practical implementation. *Neuromorphic Computing and Engineering, 2*(3), 032002. https://doi.org/10.1088/2634-4386/ac7db7

Doan, N. A. K., Polifke, W., & Magri, L. (2020). Physics-informed echo state networks. *Journal of Computational Science, 47*, 101237. https://doi.org/10.1016/j.jocs.2020.101237

Elsworth, S., Güttel, S. (2020). Time Series Forecasting Using LSTM Networks: A Symbolic Approach.

Goodfellow, I., Bengio, Y., Courville, A. (2016). Deep Learning. MIT Press. http://www.deeplearningbook.org

Hamayel, M. J., & Owda, A. Y. (2021). A novel cryptocurrency price prediction model using gru, lstm and bi-lstm machine learning algorithms. *AI, 2*(4), 477–496. https://doi.org/10.3390/ai2040030

Haykin, S. S. (2009). *Neural Networks and Learning Machines*. Pearson International Edition. Pearson. https://books.google.de/books?id=KCwWOAAACAAJ

Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Hyndman, R., & Athanasopoulos, G. (2018). *Forecasting: Principles and Practice* (2nd ed.). Australia: OTexts.

Jaeger, H. (2001). The"echo state" approach to analysing and training recurrent neural networks.

Jaeger, H. (2002). Adaptive nonlinear system identification with echo state networks. In *Proceedings of the 15th International Conference on Neural Information Processing Systems*. NIPS'02, pp. 609–616. MIT Press, Cambridge, MA, USA.

Jaeger, H. (2021). A tutorial on training recurrent neural networks, covering bptt, rtrl, ekf and the "echo state network" approach. https://api.semanticscholar.org/CorpusID:237370083

Jaeger, H. (2007). Echo state network. *Scholarpedia, 2*(9), 2330.

Jaeger, H., & Haas, H. (2004). Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *Science, 304*(5667), 78–80.

Kingma, D. P., Ba, J., Adam, A. (2017). Method for Stochastic Optimization.

Kumar, K. (2023). Machine learning in parameter estimation of nonlinear systems. arXiv preprint arXiv:2308.12393

Kumar, K. (2023). Reservoir computing in epidemiological forecasting: Predicting chicken pox incidence. medRxiv https://doi.org/10.1101/2023.04.24.23289018

Le, T.-H., Le, A. T., & Le, H.-C. (2021). The historic oil price fluctuation during the covid-19 pandemic: What are the causes? *Research in International Business and Finance, 58*, 101489. https://doi.org/10.1016/j.ribaf.2021.101489

Li, X., Ma, X., Xiao, F., & Xiao, C. (2022). Time-series production forecasting method based on the integration of bidirectional gated recurrent unit (bi-gru) network and sparrow search algorithm (ssa). *Journal of Petroleum Science and Engineering, 208*, 109309. https://doi.org/10.1016/j.petrol.2021.109309

Li, C., Tang, G., Xue, X., Saeed, A., & Hu, X. (2020). Short-term wind speed interval prediction based on ensemble gru model. *IEEE Transactions on Sustainable Energy, 11*(3), 1370–1380. https://doi.org/10.1109/TSTE.2019.2926147

Lukoševičius, M. (2012). In Montavon, G., Orr, G.B., Müller, K.-R. (eds.) *A Practical Guide to Applying Echo State Networks,* pp. 659–686. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-35289-8_36

Lukoševičius, M., & Jaeger, H. (2009). Reservoir computing approaches to recurrent neural network training. *Computer Science Review, 3*(3), 127–149. https://doi.org/10.1016/j.cosrev.2009.03.005

Maass, W., Natschläger, T., & Markram, H. (2002). Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation, 14*(11), 2531–2560. https://doi.org/10.1162/089976602760407955

Nakajima, K. (2020). Physical reservoir computing-an introductory perspective. *Japanese Journal of Applied Physics, 59*

Pathak, J., Wikner, A., Fussell, R., Chandra, S., Hunt, B. R., Girvan, M., & Ott, E. (2018). Hybrid forecasting of chaotic processes: Using machine learning in conjunction with a knowledge-based model. *Chaos: An Interdisciplinary Journal of Nonlinear Science, 28*(4), 041101. https://doi.org/10.1063/1.5028373

Rajagukguk, R. A., Ramadhan, R. A. A., & Lee, H.-J. (2020). A review on deep learning models for forecasting time series data of solar irradiance and photovoltaic power. *Energies*. https://doi.org/10.3390/en13246623

Ren, B., & Ma, H. (2022). Global optimization of hyper-parameters in reservoir computing. *Electronic Research Archive, 30*(7), 2719–2729. https://doi.org/10.3934/era.2022139

Robbins, H. E. (1951). A stochastic approximation method. *Annals of Mathematical Statistics, 22*, 400–407.

Ruder, S. (2017). An overview of gradient descent optimization algorithms.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature, 323*, 533–536. https://doi.org/10.1038/323533a0

Sezer, O. B., Gudelek, M. U., & Ozbayoglu, A. M. (2020). Financial time series forecasting with deep learning: A systematic literature review: 2005–2019. *Applied Soft Computing, 90*, 106181. https://doi.org/10.1016/j.asoc.2020.106181

Sheng, C., Zhao, J., Wang, W., & Liu, Q. (2014). Echo state network based prediction intervals estimation for blast furnace gas pipeline pressure in steel industry. *IFAC Proceedings Volumes, 47*(3), 1041–1046. https://doi.org/10.3182/20140824-6-ZA-1003.00837. 19th IFAC World Congress.

Shih, S.-Y., Sun, F.-K., Lee, H.-y. (2019). Temporal Pattern Attention for Multivariate Time Series Forecasting.

Siami-Namini, S., Tavakoli, N., Siami Namin, A. (2018). A comparison of arima and lstm in forecasting time series. In *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA),* pp. 1394–1401 . https://doi.org/10.1109/ICMLA.2018.00227

Tanaka, G., Yamane, T., Héroux, J. B., Nakane, R., Kanazawa, N., Takeda, S., Numata, H., Nakano, D., & Hirose, A. (2019). Recent advances in physical reservoir computing: A review. *Neural Networks, 115*, 100–123. https://doi.org/10.1016/j.neunet.2019.03.005

Tao, Q., Liu, F., Li, Y., & Sidorov, D. (2019). Air pollution forecasting using a deep learning model based on 1d convnets and bidirectional gru. *IEEE Access, 7*, 76690–76698. https://doi.org/10.1109/ACCESS.2019.2921578

Trouvain, N., Pedrelli, L., Dinh, T. T., & Hinaut, X. (2020). Reservoirpy: An efficient and user-friendly library to design echo state networks. In I. Farkaš, P. Masulli, & S. Wermter (Eds.), *Artificial Neural Networks and Machine Learning - ICANN 2020* (pp. 494–505). Cham: Springer.

Tseng, F.-M., Yu, H.-C., & Tzeng, G.-H. (2002). Combining neural network model with seasonal time series arima model. *Technological Forecasting and Social Change, 69*(1), 71–87. https://doi.org/10.1016/S0040-1625(00)00113-X

Valencia, C. H., Vellasco, M. M. B. R., & Figueiredo, K. (2023). Echo state networks: Novel reservoir selection and hyperparameter optimization model for time series forecasting. *Neurocomputing, 545*, 126317. https://doi.org/10.1016/j.neucom.2023.126317

Wang, W.-J., Tang, Y., Xiong, J., & Zhang, Y.-C. (2021). Stock market index prediction based on reservoir computing models. *Expert Systems with Applications, 178*, 115022. https://doi.org/10.1016/j.eswa.2021.115022

Weerakody, P. B., Wong, K. W., Wang, G., & Ela, W. (2021). A review of irregular time series data handling with gated recurrent neural networks. *Neurocomputing, 441*, 161–178. https://doi.org/10.1016/j.neucom.2021.02.046

Wikipedia contributors: Echo state network — Wikipedia, The Free Encyclopedia. [Online; accessed 27-December-2023] (2023). https://en.wikipedia.org/w/index.php?title=Echo_state_network&oldid=1185484546

Yperman, J., Becker, T. (2017). Bayesian optimization of hyper-parameters in reservoir computing.

Zhang, G. P. (2003). Time series forecasting using a hybrid arima and neural network model. *Neurocomputing, 50*, 159–175. https://doi.org/10.1016/S0925-2312(01)00702-0

Zhang, X., Liang, X., Zhiyuli, A., Zhang, S., Xu, R., & Wu, B. (2019). At-lstm: An attention-based lstm model for financial time series prediction. *IOP Conference Series: Materials Science and Engineering, 569*(5), 052037. https://doi.org/10.1088/1757-899X/569/5/052037